

תרגיל בית 4

מילונים

הנחיות כלליות:

• קראו היטב את השאלות והקפידו שהתכניות שלכם פועלות בהתאם לנדרש.

• את התרגיל יש לפתור לבד!

• הקפידו על כללי ההגשה המפורסמים באתר. בפרט, יש להגיש את כל השאלות יחד בקובץ `ex1_012345678.py` המצורף לתרגיל, לאחר החלפת הספרות 012345678 במספר ת.ז שלכם, כל 9 הספרות כולל ספרת הביקורת.

• מועד אחרון להגשה: כמפורסם באתר.

• בדיקה עצמית: כדי לוודא את נכונותן ואת עמידותן של התוכניות לקלטים שגויים, בכל שאלה, הריצו את תוכניתכם עם מגוון קלטים שונים, אלה שהופיעו כדוגמאות בתרגיל וקלטים נוספים עליהם חשבתם (וודאו כי הפלט נכון וכי התוכנית אינה קורסת).

• היות ובדיקת התרגילים עשויה להיות אוטומטית, יש להקפיד על פלטים מדויקים על פי הדוגמאות (עד לרמת הרווח)

• אופן ביצוע התרגיל: בתרגיל זה עליכם להשלים את הקוד בקובץ המצורף.

• אין לשנות את שמות המשתנים שכבר מופיעים בקובץ השלד של התרגיל. יש לעבוד עם המשתנים שמופיעים בשלד התרגיל. על הקוד של כל שאלה לעבוד ולספק את התוצאה הדרושה עבור קלט שיוזן במשתנים שמופיעים בשלד (יש להחליף את סימני השאלה שמופיעים ליד שמות משתני הקלט בערכים לבחירתכם ולהגיש את התרגיל ללא סימני השאלה). יחד עם זאת, אתם רשאים להוסיף משתנים נוספים כראות עינכם.

• אין למחוק את ההערות שמופיעות בשלד.

שאלה 1:

ממשו את הפונקציה `lists_to_simple_dict(names, dates)` אשר מקבלת רשימה של שמות (ללא חזרות) ורשימה של תאריכים באותו האורך, ומחזירה מילון הממפה לכל שם את התאריך שלו.

תאריך זה מייצג תאריך של בדיקת קורונה שלילית.

הערות:

- ניתן להניח שרשימות הקלט הן באותו אורך, אך הן עשויות להיות ריקות (במקרה זה יש להחזיר מילון ריק).
- `names` היא רשימה של מחרוזות המייצגות שמות ו `dates` היא רשימה של רשימות באורך 3 המייצגות תאריך (במיקום ה-0 השנה, במיקום ה-1 החודש ובמיקום ה-2 היום). לדוגמא 01/12/2021 ייוצג ע"י הרשימה: [1, 12, 2021]. כל רשימה באורך 3 המייצגת תאריך ברשימה `dates` יהיו תאריך בדיקה של האדם ששמו נמצא במיקום המקביל ברשימה `names` כלומר התאריך במיקום `i` כלשהו ברשימה `dates` יהיו תאריך הבדיקה של האדם ששמו מאוחסן במיקום `i` ברשימה `names`).
- במילון המוחזר כל מפתח יהיה מחרוזת המייצגת שם, וכל ערך יהיה רשימה המייצגת את התאריך (רשימה באורך 3) שמתאים לשם זה.

דוגמת הרצה:

```
>>> names = ['Alon', 'Michal', 'Yael', 'Ofir']
>>> dates = [[2021, 12, 9], [2020, 11, 8], [2021, 12, 8],
[2021, 1, 1]]
>>> dic = lists_to_simple_dict(names, dates)
>>> print (dic)

{'Alon': [2021, 12, 9], 'Michal': [2020, 11, 8], 'Yael':
[2021, 12, 8], 'Ofir': [2021, 1, 1]}
```

שאלה 2:

ממשו את הפונקציה `lists_to_complex_dict(names, dates)` אשר מקבלת רשימה של שמות (ייתכנו חזרות) ורשימה של תאריכים (כפי שפורט בשאלה 1) באותו האורך, ומחזירה מילון הממפה לכל שם את התאריך של בדיקת הקורונה השלילית הכי עדכנית שלו (כלומר מבין כל התאריכי המתאימים - את הכי מאוחר מביניהם).

בסעיף זה, ייתכן ואותו שם יופיע מספר פעמים ברשימה `names`.

- ניתן להניח שרשימות הקלט הן באותו אורך, אך הן עשויות להיות ריקות (במקרה זה יש להחזיר מילון ריק)
- `names` היא רשימה של מחרוזות המייצגות שמות ו `dates` היא רשימה של רשימות המייצגות תאריך בדיקה. כל תאריך ברשימה `dates` הינו, תאריך בדיקה של האדם ששמו נמצא במיקום המקביל ברשימה `names` (כלומר מספר הטלפון במיקום `i` כלשהו ברשימה `dates` הינו, תאריך בדיקה של האדם ששמו מאוחסן במיקום `i` ברשימה `names`).
- במילון המוחזר כל מפתח יהיה מחרוזת המייצגת שם, וכל ערך יהיה רשימה באורך 3 המייצגת את תאריך בדיקת הקורונה השלילית המאוחרת ביותר של אותו אדם.

דוגמת הרצה:

```
>>> names =
['Alon', 'Michal', 'Alon', 'Yael', 'Ofir', 'Ofir', 'Ofir']
>>> dates = [[2021,12,9], [2020,11,8], [2021,12,8],
[2021,1,1], [2021,2,2], [2021,1,4], [2021,2,3]]
>>> d=lists_to_complex_dict(names, dates)
>>> print(d)

{'Alon': [2021, 12, 9], 'Michal': [2020, 11, 8], 'Yael':
[2021, 1, 1], 'Ofir': [2021, 2, 3]}
```

שאלה 3:

ממשו פונקציה בשם `find_substring_locations(s, k)` המקבלת מחרוזת ואורך של תת-מחרוזת כמספר

שלם ומחזירה את הפלטים הבאים: `s`:

- המפתחות הם כל תתי המחרוזות של המחרוזת `s` (אוסף רצוף של תווים מתוך `s` באורך `k`)
- הערך המתאים לכל מפתח הוא כמות הפעמים שתת מחרוזת זו מופיעה במחרוזת בגדולה.
- יש להחזיר **שני** פלטים: הראשון- את המילון מתואר, והשני - תת המחרוזת השכיחה ביותר.
- במידה ויש כמה תתי מחרוזת בעלי אותה תדירות (מירבית) יש להחזיר את הראשון שמופיע במחרוזת (רמז - ביחנו את פונקציית המיון של מילונים במקרה של ריבוי תתי מחרוזות)
- ניתן להניח כי $k > 0$ אם k גדול מאורך המחרוזת יש להחזיר מילון ריק. ושאינו גדול מאורך המחרוזת.

דוגמאות:

```
>>> txt = 'abdsfabababsfasf'
>>> res = find_substring_location(txt,2)
>>> print(res[0])
>>> print(res[1])
{'ab': 4, 'bd': 1, 'ds': 1, 'sf': 3, 'fa': 2, 'ba': 2, 'bs': 1, 'as': 1}
ab
```

```
>>> res = find_substring_location(txt,3)
>>> print(res[0])
>>> print(res[1])
{'abd': 1, 'bds': 1, 'dsf': 1, 'sfa': 2, 'fab': 1, 'aba': 2, 'bab': 2, 'abs': 1, 'bsf': 1, 'fas': 1, 'asf': 1}
sfa
```

```
>>> res = find_substring_location(txt,1)
>>> print(res[0])
>>> print(res[1])
{'a': 5, 'b': 4, 'd': 1, 's': 3, 'f': 3}
a
```

```

>>> txt = "abcabcbaaaccscscscffgf"
>>> res = find_substring_location(txt,2)
>>> print(res[0])
>>> print(res[1])
{'ab': 2, 'bc': 2, 'ca': 1, 'cb': 1, 'ba': 1, 'aa': 2, 'ac':
1, 'cc': 3, 'cs': 3, 'sc': 3, 'cf': 1, 'ff': 1, 'fg': 1, 'gf':
1}
cc
>>> res = find_substring_location(txt,3)
>>> print(res[0])
>>> print(res[1])
{'abc': 2, 'bca': 1, 'cab': 1, 'bcb': 1, 'cba': 1, 'baa': 1,
'aaa': 1, 'aac': 1, 'acc': 1, 'ccc': 2, 'ccs': 1, 'csc': 3,
'scs': 2, 'scf': 1, 'cff': 1, 'ffg': 1, 'fgf': 1}
csc

```

שאלה 4:

בכיתה ראינו ייצוג חסכוני בזיכרון עבור מטריצות דלילות שהינן מטריצות שמרבית איבריהן הם אפס (sparse matrix). במקום להחזיק את כל איברי המטריצה בזיכרון כרשימה של רשימות, נשתמש במילון בו נשמור רק את האיברים ששונים מאפס (כערכי המילון) ואת מיקומם במטריצה (כמפתחות המילון). עבור כל איבר במטריצה ששונה מאפס, נשמור במילון כמפתח את מיקום האיבר (tuple הכולל את אינדקס השורה ואת אינדקס העמודה) וכערך נשמור את האיבר במטריצה. למשל, בהינתן המטריצה הדלילה באה (המיוצגת ע"י רשימה של רשימות):

```

[[0, 0, 0, 1, 0],
 [0, 0, 0, 0, 0],
 [0, 2, 0, 0, 0],
 [0, 0, 0, 0, 0],
 [0, 0, 0, 3, 0]]

```

נוכל לייצג אותה כמילון כך :

```
{(0, 3): 1, (2, 1): 2, (4, 3): 3}
```

למשל, המפתח (0,3) הינו tuple המייצג את המיקום שורה 0 ועמודה 3 וערך האיבר שם הוא 1.

מממש את הפונקציה `mul_sparse_matrices(mat1,mat2)` אשר מקבלת שני מילונים המייצגים מטריצות דלילות באותם הממדים, ומחזירה מילון המייצג את מטריצת הכפל. במטריצת הפלט, כל איבר יהיה הכפל של האיברים במיקום המקביל במטריצות הקלט.

הערות:

- ניתן להניח ששתי מטריצות תקינות ושהינן בעלות מימדים זהים.
- רמז - חישבו אילו איברים יהיו שונים מ-0 במטריצה הסופית.

דוגמת הרצה:

```
>>> mat1 = {(0,1): 2, (2,5): 1, (3,3): -10}
>>> mat2 = {(0,0): 5, (3,3): 8}
>>> print (mul_sparse_matrices(mat1, mat2))
{(3, 3): -80}
```

```
>>> mat1 = {(0,0): 2, (2,5): 1, (3,3): -10}
>>> mat2 = {(0,0): 5, (3,3): 8}
>>> print (mul_sparse_matrices(mat1, mat2))
{(0, 0): 10, (3, 3): -80}
```

בהצלחה !